

ECMA-55 BASIC

Language Manual

Augusto Manzano

2026

This manual documents the BASIC language exactly as defined by the ECMA-55 standard (1st edition, January 1978). The same standard described under the designation ANSI X3.60-1978, informally known as “Minimal BASIC”.

The interpreter (`ecma55basic`) implements only what the standard defines, with no additions. It’s teaching material for getting to know BASIC as it was born, before every manufacturer bolted on its own extensions.

Contents

1. Overview
2. Lexical elements
3. Variables and arrays
4. Expressions and operators
5. Built-in functions
6. Statement reference
7. Errors and exceptions
8. The interactive environment

How to use this manual

If you’ve never programmed in BASIC, read the chapters in order. If you already know the language (or a variant of it, like GW-BASIC or QBasic) and just want to check a specific piece of syntax, go straight to chapter 6 (statements) or 5 (functions).

Every code example in this manual actually runs on this interpreter. Try it yourself, either by typing `ecma55basic your-program.bas` or by pasting the code into the interactive environment (chapter 8).

1. Overview

1.1. What Minimal BASIC is

In 1978, when BASIC was already running on dozens of different computers each manufacturer with its own version, incompatible with the rest two standards bodies, ANSI (American) and ECMA (European), published a common standard describing the minimum core that any BASIC implementation would need to offer to actually be called “BASIC”. That standard is ECMA-55 (word-for-word identical to ANSI X3.60-1978).

“Minimum” is the key word here: the standard didn’t try to describe *the* BASIC of any specific manufacturer, but rather the smallest subset common to all of them, a foundation on top of which each manufacturer could build its own extensions while still being, technically, compliant with the standard. That’s why the language is informally known as “Minimal BASIC”.

This explains why the language, seen through today’s eyes, looks so lean: no string functions beyond the bare minimum, no WHILE, no records, no programmer-defined types. This isn’t a poorly designed limitation, it’s the deliberate definition of a common floor.

1.2. Why an interpreter strictly adherent to the standard

This project implements exactly what the standard defines, and nothing beyond that. Its purpose is to serve as teaching material faithful to what BASIC historically was, before each manufacturer added whatever features it wanted. A student who learns with this interpreter learns the standard, not a variant of it..

1.3. Building and running

The project uses a simple Makefile, from inside the project folder.

Linux

```
make          - builds the interpreter (produces the ecma55basic executable)
make test     - builds and runs the project's own test suite (lexer, parser, interpreter)
make nbs      - builds and runs the NBS conformance battery
make clean    - removes the files generated by the build
```

Windows

```
mingw32-make          - builds the interpreter (produces the ecma55basic executable)
mingw32-make test     - builds and runs the project's own test suite (lexer, parser, interpreter)
mingw32-make nbs      - builds and runs the NBS conformance battery
mingw32-make clean    - removes the files generated by the build
```

Requires a C++17 compiler (g++, for example) and the make/mingw32-make utility. No other dependencies.

Once built, there are two ways to use the interpreter:

```
./ecma55basic          - opens the interactive environment (see chapter 8)
./ecma55basic program.bas - runs a program file directly
```

1.4. The structure of a program

A Minimal BASIC program is a sequence of numbered lines, one after another, in strictly ascending numeric order. Each line contains exactly one statement. The last line of the program, and only that one, must be an END statement.

```
10 PRINT "HELLO, WORLD"
20 END
```

The line number (here, 10 and 20) isn't decoration: it's what defines the program's execution order (which runs, by default, in the lines' numeric order), and it's what the branching statements (GOTO, GOSUB, IF . . . THEN) use to say where to jump to. A line number can have 1 to 4 digits (from 1 to 9999).

By convention, not because the standard requires it, most programs number their lines in steps of 10 (10, 20, 30...), leaving room to insert new lines later without having to renumber everything.

1.5. Uppercase and lowercase

The standard defines BASIC as being written entirely in uppercase letters: reserved words, variable names, everything. This interpreter accepts lowercase in the source code as a practical convenience and normalizes it to uppercase internally, but the content of a string between quotes ("Like this") preserves exactly the upper/lowercase you wrote. After all, that's data, not code.

1.6. What comes next

Chapter 2 details the language's "alphabet", how a number, a string, or a variable name are recognized. If you're already familiar with BASIC's general syntax, feel free to skip straight to chapter 6 (statement reference).

2. Lexical elements

This chapter describes the smallest units that make up a program: the language's "alphabet". Before a statement makes sense, the interpreter needs to recognize, character by character, where a number ends and a reserved word begins, where a string ends and something else begins. That recognition is what this chapter describes.

2.1. Character set

The standard defines the language over an alphabet of uppercase letters (A–Z), digits (0–9), and a set of punctuation symbols and operators. This interpreter accepts lowercase in the source code as a practical convenience (normalized to uppercase), but the content of a string between quotes preserves exactly what was written: uppercase, lowercase, spaces, everything.

Each program line has at most 72 characters.

2.2. Numbers

A number is written in decimal notation, with a period (not a comma) separating the integer part from the fractional part, and an optional exponent introduced by E:

```
500
-21.
.255
1E10
5E-1
.4E+1
```

Notice that `-21.` is valid (the period with nothing after it), just like `.255` (nothing before the period). A number's sign, when it appears alone in front of an expression or after an operator, is treated as part of the expression and not of the number itself.

Internally, numbers are stored as double-precision floating point (C++'s `double` type), which guarantees far more digits of precision than any Minimal BASIC program should ever need.

2.3. Strings

A string is text between double quotes:

```
"HELLO, WORLD"
"Can have lowercase, numbers 123, and punctuation!"
```

There's no escape mechanism, a string cannot contain a quote inside it, because the first quote that appears closes the string. There's no length limit for a string *constant* (written directly in the code), but a string variable (section 3.2) has a guaranteed minimum of 18 characters.

2.4. Identifiers (names)

The standard defines very restricted names, quite different from modern languages:

Form	Means	Examples
One letter	Simple numeric variable	A, X, Z
One letter + one digit	Simple numeric variable	A1, X9, B0
One letter + \$	String variable	A\$, N\$
FN + one letter	Function defined with DEF FN (chapter 4)	FNA, FNF

There are no descriptive names like TOTAL or COUNTER — the most the language allows is one letter and, optionally, one digit or the \$. This is a real limitation of the standard, not of this interpreter: program with short names, the way it was done back then.

An array (chapter 3) uses a single letter as its name, never letter+digit. DIM A9(10) is invalid; DIM A(10) is valid.

2.5. Reserved words

The following 24 words have a fixed meaning in the language and cannot be used as a variable name (which, given that variable names have at most two letters/digits, would never be a conflict anyway):

```
BASE DATA DEF DIM END FOR GO IF INPUT LET
NEXT ON OPTION PRINT RANDOMIZE READ REM RESTORE RETURN
STEP STOP SUB THEN TO
```

GOTO and GOSUB aren't separate words, they're the glued spelling of GO TO and GO SUB (the standard allows writing them with or without a space between GO and TO/SUB). The eleven built-in functions (chapter 5) and PRINT's TAB are also reserved, but unlike the 24 words above. They can appear glued to a parenthesis, with no space required before them (SQR(4), TAB(10)).

Every reserved word (from the 24 main ones) needs a space before it and, if it's not the end of the line, a space after it. 100LET X=5 is invalid; it needs to be 100 LET X=5.

2.6. Comments

The REM statement (chapter 6) marks the rest of the line as a comment: free text, ignored by the interpreter:

```
100 REM THIS PROGRAM CALCULATES THE AVERAGE OF THREE GRADES
```

There's no end-of-line comment (like ' or //) after a statement, REM needs to be the whole statement on the line.

2.7. Operators and punctuation

```
+ - * / ^      arithmetic operators (chapter 4)
= <> < <= > >= relational operators (chapter 4)
( )           parentheses for expressions, arrays, and functions
, ;          separators (PRINT, variable lists, DIM)
```

Chapter 4 explains the precedence between the arithmetic operators and how they behave.

3. Variables and arrays

3.1. Two types, only two

The language has exactly two data types: **number** (floating point) and **string** (text). There are no integers as a separate type, no booleans (a comparison like `X > 3` produces a number: 1 for true, 0 for false), no programmer-defined types.

A variable's type is determined by its own spelling `A` is always numeric, `A$` is always a string never by the value you assign to it.

3.2. Numeric and string variables

A simple numeric variable is a letter, optionally followed by a digit (`A`, `B3`, `Z9`, see section 2.4). A string variable is a letter followed by `$` (`A$`, `N$`).

```
100 LET X = 5
110 LET N$ = "MARY"
```

Every variable must be given a value before it's read. Reading a variable that was never assigned is a fatal error, which stops the program (chapter 7). A design decision that follows the standard's explicit recommendation (section 7.6), instead of the common custom of silently assuming zero. This is deliberate: a "forgotten" value is usually a sign of a bug in the program, and it's better to catch it right away than to carry an incorrect zero forward without noticing.

A string stored in a variable has a guaranteed minimum of 18 characters. Trying to store a longer string is a fatal error with `LET` and `READ`, but not with `INPUT` (which, in that case, simply asks for the answer again, without bringing down the program).

3.3. Arrays

An array is a "table" of numeric values, indexed by an integer (or two, for two-dimensional tables). An array's name is always a single letter, never letter+digit.

An array doesn't need to be explicitly declared: if you use `A(3)` without ever having given a `DIM A( . . . )` before, the array `A` is automatically born with indices from 0 to 10 (11 slots). For a larger array, or to make the size explicit in the code, use `DIM`:

```
100 DIM A(100)
110 LET A(50) = 3.14
120 PRINT A(50)
```

This creates an array `A` with indices from 0 to 100 (101 slots). `DIM` must come before the array's first use, and the same array can only be dimensioned once.

A two-dimensional array uses two indices, separated by a comma:

```
100 DIM B(10, 20)
110 LET B(5, 12) = 7
```

A letter can only be one thing: a simple variable, a one-dimension array, or a two-dimension array. Never two of these at the same time in the same program. Using A as a simple variable on one line and as A(3) on another is an error.

Just like simple variables, an array element must be given a value before it's read.

3.4. OPTION BASE — changing where indices start

By default, an array's indices start at 0. The `OPTION BASE 1` statement, if used, changes this: indices start at 1 instead, and an array with no explicit DIM is born with indices from 1 to 10 (10 slots, not 11).

```
100 OPTION BASE 1
110 DIM A(10)
120 LET A(1) = 100
```

`OPTION BASE` can only appear once in the program, and must come before any DIM or array use, including before an array implicitly created by its first use.

3.5. Where variables “live”

Every variable and every array is **global**: visible from any point in the program, start to finish. The one exception is the parameter of a function defined with `DEF FN` (chapter 4), which is local to that definition and doesn't interfere with a variable of the same name used elsewhere in the program.

4. Expressions and operators

4.1. Arithmetic operators and precedence

```
^      exponentiation
* /    multiplication and division
+ -    addition and subtraction
```

Top to bottom, highest to lowest precedence exactly like ordinary math. Parentheses change the order normally:

```
PRINT 5 + 3 * 2
PRINT (5 + 3) * 2
```

The first line prints 11 (3*2 first, then +5); the second prints 16.

A detail that surprises people coming from other languages: exponentiation (^) is **left-associative**, not right. That is, A^B^C is computed as $(A^B)^C$, not as $A^{(B^C)}$. In most modern languages (Python, C, etc.) it's the other way around — if that's where you're coming from, pay attention to this one.

```
PRINT 2^3^2
```

The result is $(2^3)^2 = 8^2 = 64$, **not** $2^{(3^2)} = 2^9 = 512$.

Operators of the same precedence (* with /, or + with -) are also evaluated left to right: $10 - 3 - 2$ is $(10-3)-2 = 5$, not $10-(3-2) = 9$.

4.2. Special arithmetic cases

The language handles a few cases that, in pure math, don't have a single answer, in a specific and predictable way:

- $0 \wedge 0$ equals 1.
- Dividing by zero doesn't stop the program, the result becomes "infinity" (with the numerator's sign), and execution continues.
- A number raised to an exponent so large that the result exceeds the computer's capacity also becomes "infinity", without stopping the program.
- Raising a **negative** number to a **non-integer** exponent (for example, $(-4) \wedge 0.5$, which would be a square root of a negative number) is a fatal error, because the result would be a complex number, outside what the language represents.

Chapter 7 details the difference between an error that stops the program and one that just substitutes the value and moves on.

4.3. Relational operators

```
= equal
<> not equal
< less than
<= less than or equal
> greater than
>= greater than or equal
```

Used inside an IF (chapter 6) to test a condition. The result is always a number: 1 if the comparison is true, 0 if it's false.

```
100 IF X > 3 THEN 200
```

Numbers are compared numerically and strings are compared character by character, but **strings can only use = and <>**, never <, <=, >, or >=. Comparing a number with a string is invalid.

4.4. Built-in and defined functions

An expression can call one of the eleven built-in math functions (chapter 5) or a function created with DEF FN (chapter 6):

```
100 PRINT SQR(16)
110 DEF FNF(X) = X^2 + 1
120 PRINT FNF(3)
```

Line 100 (built-in function) prints 4; line 120 (defined function) prints 10.

4.5. String expressions

Unlike numbers, a string expression is quite limited: it can only be a quoted constant or a string variable, **there's no concatenation operator** or any other string operation in the language. If you need to build up text from pieces, that has to be done piece by piece, for example with several items in the same PRINT (chapter 6).

5. Built-in functions

The language has exactly eleven ready-made math functions. All of them take a single numeric argument in parentheses except for RND, the only one with no argument at all. Unlike the other reserved words, a function's name can come glued to the parenthesis, with no space: SQR(4) is valid.

Function	What it computes
ABS(X)	Absolute value of X
ATN(X)	Arctangent of X, in radians
COS(X)	Cosine of X (X in radians)
EXP(X)	<i>e</i> raised to X
INT(X)	Largest integer not greater than X (rounds down)
LOG(X)	Natural logarithm of X
RND	A pseudo-random number between 0 (inclusive) and 1 (exclusive)
SGN(X)	1 if X is positive, -1 if negative, 0 if X is zero
SIN(X)	Sine of X (X in radians)
SQR(X)	Square root of X
TAN(X)	Tangent of X (X in radians)

Details worth noting

INT rounds down, not to the nearest integer. INT(3.9) equals 3, and INT(-3.1) equals -4 (the largest integer that is *not greater* than -3.1, i.e., further toward the negative side). If you need to round to the nearest integer, you have to build the calculation yourself. For example, INT(X + 0.5) for positive values.

SQR of a negative number is a fatal error. The language has no complex number type, so there's no possible result. The same applies to LOG of zero or of a negative number.

RND without RANDOMIZE always repeats the same sequence of numbers, every time you run the program. This is intentional: it helps test and debug a program that uses random numbers, since the result is always the same from one run to the next. To get a different sequence on every run, use the RANDOMIZE statement (chapter 6) before the first RND.

```
100 RANDOMIZE
110 PRINT INT(RND * 6) + 1
```

This prints a number from 1 to 6, like a die.

6. Statement reference

This chapter documents, one by one, every statement in the language. Each section shows the syntax, explains what the statement does, and gives a runnable example.

6.1. LET — assignment

LET variable = expression

Stores the value of an expression in a variable (simple, array, or string). The left side and the right side must be the same type, you can't assign a string to a numeric variable, or vice versa.

```
100 LET X = 5
110 LET Y = X * 2 + 1
120 LET N$ = "MARY"
130 LET A(3) = Y
```

6.2. PRINT — output

PRINT [item] [(,|;) item]...

Prints a sequence of items, numeric expressions, strings, or TAB(expression) to position the cursor at a specific column separated by commas or semicolons.

- A **comma** advances the cursor to the next *print zone* (the line is divided into fixed-width zones, like columns of a table).
- A **semicolon** doesn't insert any space at all, it just separates one item from the next.
- A PRINT by itself, with nothing after it, prints a blank line.
- If the list ends in a comma or semicolon, the cursor **doesn't** jump to the next line, the next PRINT continues from where this one stopped.

```
100 PRINT "HELLO, WORLD"
110 PRINT "A ="; A
120 PRINT "COLUMN 1", "COLUMN 2"
130 PRINT TAB(20); "ALIGNED AT COLUMN 20"
```

On line 110, the semicolon doesn't insert any extra space before A's value; on line 120, the comma sends each item to a different print zone.

A printed number always comes with a space (if positive) or a minus sign (if negative) before it, and a space after it. That's how the standard defines numeric formatting, so numbers stay visually separated from each other even with no explicit separator.

6.3. INPUT — user input

INPUT variable [, variable]...

Pauses the program, shows the ? prompt, and waits for the user to type a value (or several, separated by commas) for each variable in the list.

```
100 PRINT "WHAT IS YOUR NAME?"
110 INPUT N$
120 PRINT "HELLO, "; N$
```

If the user types the wrong number of values, or text where a number was expected, the program **doesn't** stop, it warns and asks for the answer again. This is an important difference from LET and READ, where the same kind of error is fatal.

6.4. READ and DATA — constant data

```
READ variable [, variable]...  
DATA value [, value]...
```

DATA declares a list of constant values, fixed in the program's code. READ reads, one at a time, the next values from that list and stores them in the given variables like an automatic INPUT, whose "typed values" are already written into the program.

```
100 READ A, B, C  
110 PRINT A + B + C  
120 DATA 10, 20, 30
```

Notice that DATA can appear on any line of the program, it doesn't need to be near the READ that will consume it. All the DATA in the program forms a single list, in the order it appears.

A value in DATA can be a number, a quoted string, or an *unquoted* string (only letters, digits, spaces, +, -, and .):

```
100 DATA 3.14159, PI, 5E-10, ", "
```

Unlike INPUT, if READ doesn't find enough data, or finds data of the wrong type (for example, trying to read a string into a numeric variable), the program **stops**, it's a fatal error.

6.5. RESTORE — restart reading

```
RESTORE
```

Resets the READ pointer back to the first DATA value in the program, allowing the same list to be read again from the start.

```
100 DATA 1, 2, 3  
110 READ A, B, C  
120 PRINT A; B; C  
130 RESTORE  
140 READ A  
150 PRINT A
```

The last line prints 1 again, RESTORE made the READ on line 140 reread the first DATA value.

6.6. DIM — declaring an array

```
DIM name(bound) [, name(bound)]...  
DIM name(bound, bound) [, ...]...
```

Explicitly declares an array's size, see section 3.3 for a full explanation. DIM must come before the first use of the array it declares, and each array can only be dimensioned once.

```
100 DIM GRADES(50), TABLE(10, 10)
```

6.7. OPTION BASE — changing the arrays' starting index

```
OPTION BASE 0
OPTION BASE 1
```

Changes the arrays' starting index from 0 (the default) to 1. Can only appear once in the program, and must come before any DIM or array use, see section 3.4.

6.8. DEF — defining a function

```
DEF FNx = expression
DEF FNx(parameter) = expression
```

Creates a function named FNx (x is a single letter), reusable in any expression in the program from that point on. A function can have zero or one parameter.

```
100 DEF FNSQUARE(X) = X * X
110 PRINT FNSQUARE(5)
120 DEF FNPI = 3.14159
130 PRINT FNPI
```

Line 110 prints 25; line 130 prints 3.14159.

A function must be defined **before** the first line that uses it, and cannot call itself (neither directly nor indirectly, through another function). The language has no recursion. A function's parameter is local to its definition: if the parameter is called X, that X is only valid inside the function's calculation, without affecting (or being affected by) an X variable used elsewhere in the program.

6.9. GOTO and GOSUB — unconditional branching

```
GOTO line
GOSUB line
RETURN
```

GOTO line (can also be written GO TO line, with a space) transfers execution straight to the given line number, with no condition at all.

GOSUB line does the same thing, but *remembers* where it came from: when the program hits a RETURN, it goes back to the line right after the GOSUB. It's the language's subroutine mechanism, useful for a piece of calculation repeated at several points in the program.

```
100 LET A = 4 : GOTO 200
...
200 PRINT A
```

(The : in the example above is only illustrative, the standard doesn't allow two statements on the same line; it would need to be split into two lines.)

```
100 LET N = 5
110 GOSUB 500
120 PRINT "DOUBLE IS "; R
130 STOP
500 REM SUBROUTINE: DOUBLES N, STORES IN R
510 LET R = N * 2
520 RETURN
```

A RETURN with no matching GOSUB (for example, if the program's flow "falls into" the subroutine instead of being called by GOSUB) is a fatal error. That's why the example above uses STOP before line 500, so execution doesn't fall straight into the subroutine without having been called.

6.10. IF...THEN — conditional branching

IF condition THEN line

Tests a condition (a comparison, see section 4.3) and, if it's true, transfers execution to the given line number. If it's false, execution simply continues on the next line.

```
100 INPUT X
110 IF X > 0 THEN 140
120 PRINT "NOT POSITIVE"
130 GOTO 150
140 PRINT "POSITIVE"
150 END
```

There's no ELSE and no IF...THEN...END IF block — THEN is always followed by a line number (never directly by another statement), and the "other path" is always written as the natural continuation of the program, as in the example above.

6.11. ON...GOTO — branching to several places

ON expression GOTO line [, line]...

Computes a numeric expression, rounds it to the nearest integer, and uses that number to pick which line in the list to visit: 1 picks the first, 2 the second, and so on. If the computed value is less than 1 or greater than the number of lines in the list, it's a fatal error.

```
100 PRINT "1-RED 2-GREEN 3-BLUE"
110 INPUT N
120 ON N GOTO 200, 300, 400
130 PRINT "INVALID CHOICE"
140 GOTO 100
200 PRINT "YOU CHOSE RED"
210 GOTO 500
300 PRINT "YOU CHOSE GREEN"
310 GOTO 500
400 PRINT "YOU CHOSE BLUE"
500 END
```

6.12. FOR and NEXT — counted loop

FOR variable = start TO limit [STEP increment]
... loop body ...
NEXT variable

Repeats the section between FOR and NEXT once for each value of the control variable, starting at start and going up to limit (inclusive), advancing by increment each pass. STEP is optional, if omitted, the increment is 1.

```
100 FOR I = 1 TO 5
110   PRINT I
120 NEXT I
130 END
```

This prints 1 2 3 4 5, one per line. A negative increment counts backward:

```

100 FOR I = 10 TO 1 STEP -1
110   PRINT I
120 NEXT I

```

start and limit are computed only once, on entering the loop and not on every pass. Changing the value of a variable used as the limit, inside the loop body, doesn't affect how many times it repeats.

If the condition already fails right away (for example, FOR I = 10 TO 1 with no negative STEP), **the body doesn't run even once**. The loop is tested *before* each pass, never after.

A FOR loop can contain another one inside it (nested), as long as each NEXT closes the innermost still-open FOR, without overlapping:

```

100 FOR I = 1 TO 3
110   FOR J = 1 TO 3
120     PRINT I; J
130   NEXT J
140 NEXT I

```

Two nested loops can't use the same control variable, and no statement can branch (GOTO, GOSUB, IF . . . THEN) into a loop from outside it, only from inside to outside, or from inside to inside the same loop.

6.13. RANDOMIZE

RANDOMIZE

Reseeds the sequence of pseudo-random numbers used by RND (chapter 5) from an unpredictable seed, so each run of the program produces a different sequence.

6.14. STOP and END

STOP
END

STOP stops the program's execution immediately, at any point. It can appear as many times as needed.

END also stops execution, but with one more rule: **every program must have exactly one END, and it must be the last line of the program**. Think of END as marking "the program's text ends here" (even if actual execution stops earlier, due to a STOP or an error), and STOP as "stop executing here", with no relation to where the program's text ends.

6.15. REM — comment

REM free text

Marks the rest of the line as a comment: it's not executed. See section 2.6.

```

100 REM THIS PROGRAM CALCULATES THE AREA OF A CIRCLE
110 INPUT R
120 PRINT 3.14159 * R^2

```

7. Errors and exceptions

Not every problem at run time stops the program. The standard distinguishes two kinds of abnormal situation, and this interpreter follows that distinction:

7.1. Fatal errors

Stop the program immediately, showing a message that points to the line where the problem occurred. Some examples:

- Reading a variable (or array element) before assigning it a value (section 3.2).
- Array index outside the declared bounds.
- SQR of a negative number, or LOG of zero or a negative number.
- Negative base raised to a non-integer exponent.
- RETURN with no matching GOSUB.
- ON . . . GOTO with a value outside the line list.
- GOTO, GOSUB, or IF . . . THEN pointing to a line number that doesn't exist in the program.
- Assigning (LET or READ) a string longer than 18 characters to a string variable.
- Using the same letter as both a simple variable and an array.
- A function (DEF FN) calling itself.

Most of these checks happen **before** any line of the program runs. The interpreter analyzes the whole program's structure first (for example, checking that every GOTO points to a line that really exists) and only starts executing if everything is consistent. This means that, for these cases, the program doesn't even get to print anything before being rejected.

7.2. Non-fatal situations

Not every unusual situation stops the program, some have a defined recovery, and execution continues normally:

- **Division by zero** doesn't stop the program, the result becomes "infinity" (positive or negative, matching the numerator's sign).
- **A calculation that's too large** (for example, $9999 \wedge 9999$) also becomes "infinity", without stopping anything, but if that "infinity" is later used as an array index, then it does become a fatal error (index out of range).
- $0 \wedge 0$ **is not an error**, the result is defined as 1.
- In INPUT, typing the wrong number of values, or text where a number was expected, doesn't stop the program: the interpreter warns and asks for the answer again.
- An invalid TAB argument (for example, a number less than 1) is adjusted automatically, without stopping the program.

7.3. Points the standard leaves open

The ECMA-55 standard deliberately defines some behaviors as “up to each implementation” (formally, *implementation-defined* — Appendix 4 of the standard), as long as certain minimum limits are respected. This interpreter makes specific decisions on these points, documented in detail in the comments of `src/interpreter.hpp`. In short:

Point	This interpreter’s decision
Uninitialized variable	Fatal error (the standard recommends this, but doesn’t require it)
Overflow/underflow range	double’s native range ($\sim 1\text{E}\pm 308$)
PRINT print zones	5 zones of 16 columns (80-column margin)
Number display precision	8 significant digits
INPUT prompt	?
RND without RANDOMIZE	Always the same sequence

These choices were calibrated by comparing this interpreter’s output against **bas55** (Jorge Giner Cordero), a mature implementation tested for years against the NBS conformance battery (see `tests/nbs/README.md`).

8. The interactive environment

Running `ecma55basic` with no arguments opens the interactive environment: a place to type, test, and adjust a program bit by bit, instead of writing everything at once into a file.

```
$ ecma55basic
ECMA-55 BASIC [AM-42] - v0.1.0.2609.07
(c) 2016-2018 (2026 - Revised)
Augusto Manzano - All rights reserved
Type HELP to see the environment commands.
]
```

The `]` is the “prompt”: sign that the environment is waiting for you to type something.

Important: the commands in this chapter (`RUN`, `LIST`, `NEW`, `LOAD`, `SAVE`, `LOCALIZE`, `SYSTEM`, `HELP`) are tools of the *environment*, not part of the BASIC language itself, the ECMA-55 standard doesn't define them. They're conventions common to practically every interactive BASIC of the era (the same pattern as Dartmouth BASIC, GW-BASIC, etc.).

8.1. Typing a program

A line that starts with a number enters (or replaces) a line of the program stored in the environment's memory:

```
] 10 PRINT "HELLO, WORLD"
] 20 END
```

To delete a line, type just its number, with nothing after it:

```
] 10
```

8.2. RUN, LIST, NEW

```
] RUN      runs the program typed so far
] LIST     shows the program typed so far
] NEW      clears the program from memory, to start a new one from scratch
```

8.3. Immediate mode

A line typed **without** a number in front of it runs right away, like a calculator:

```
] PRINT 2 + 2
4
] LET X = 10
] PRINT X * 3
30
```

This is useful for testing an expression or checking a variable's value quickly, without having to build a whole program.

Note: immediate mode uses its own variable space, separate from what a `RUN` uses. After running a program with `RUN`, its variables don't show up in immediate mode, and vice versa. A branching statement (`GOTO`, `FOR`, etc.) typed in immediate mode doesn't work, it only makes sense inside a numbered program.

8.4. LOAD and SAVE

```
] SAVE "my-program.bas"    writes the current program to a file
] LOAD "my-program.bas"    reads a program from a file
```

The saved file is an ordinary .bas file, which can also be run directly from the command line (ecma55basic my-program.bas), without needing to open the environment.

8.5. LOCALIZE — switching the environment's language

```
] LOCALIZE PT    environment messages in Portuguese
] LOCALIZE EN    environment messages in English (default)
```

This affects the banner, HELP, and **every** message the interpreter writes, including the lexer's, parser's, and interpreter's error messages (e.g., "variable used before being given a value"). The chosen language applies to the whole session, both in immediate mode and during a RUN.

The initial language can also be chosen on the command line, before even opening the environment:

```
ecma55basic --pt    opens the environment already in Portuguese
ecma55basic --en    opens the environment already in English (the default, even without the flag)
```

8.6. Leaving the environment

```
] SYSTEM
```

BYE and EXIT work as synonyms for SYSTEM. Closing standard input (Ctrl-D on Linux/macOS, Ctrl-Z on Windows) also ends the environment.

8.7. HELP

```
] HELP
```

Shows a quick summary of all the commands in this chapter, right in the environment, useful when you don't have this manual on hand.

HELP followed by the name of a language statement or function shows its description, syntax, and an example, the same information from chapters 5 and 6, condensed for the terminal:

```
] HELP PRINT
PRINT - Prints a list of items (numbers, strings, or TAB(n)),
separated by a comma (jumps to the next zone) or a semicolon
(no space). Alone, prints a blank line.
```

```
Syntax:
  PRINT [item] [(,|;) item]...
```

```
Example:
  100 PRINT "X ="; X
  110 PRINT "A", "B", "C"
```

Works for any of the 20 statements and 11 functions. For example, HELP SQR, HELP FOR, HELP DEF.